

Mini Project

On

Music Genre Classification

Submitted in partial fulfillment of the requirements for the award of degree of

**BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING**

by

20WH1A0542 T. Tharunika

Under the esteemed guidance of

**Ms. S. Vidyullatha
Assistant Professor**



Department of Computer Science & Engineering

BVRIT HYDERABAD

COLLEGE OF ENGINEERING FOR WOMEN

(UGC-Autonomous, NBA Accredited EEE, ECE, CSE, IT&AIML B. Tech Courses)

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Bachupally, Hyderabad – 500090

Dec, 2023

BVRIT HYDERABAD
COLLEGE OF ENGINEERING FOR WOMEN
(NBA Accredited EEE, ECE, CSE, IT&AIML B. Tech Courses)
(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)
Bachupally, Hyderabad – 500090

Department of Computer Science & Engineering



CERTIFICATE

This is to certify that the mini project entitled “**Music Genre Classification**” is a bonafide work carried out by **Ms. T. Tharunika (20WH1A0542)** in partial fulfillment for the award of B. Tech degree in **Computer Science & Engineering, BVRIT HYDERABAD College of Engineering for Women, Bachupally, Hyderabad**, affiliated to Jawaharlal Nehru Technological University Hyderabad, under my guidance and supervision. The results embodied in the project work have not been submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide
Ms. S. Vidyullatha
Assistant Professor, CSE

Head of the Department
Dr. R. Suneetha Rani

External Examiner

DECLARATION

We hereby declare that the work presented in this project entitled “**Music Genre Classification**” submitted towards completion of Project work in IV Year of B. Tech of CSE at **BVRIT HYDERABAD College of Engineering for Women**, Hyderabad is an authentic record of our original work carried out under the guidance of **Ms. S. Vidyullatha** , **Assistant Professor, Department of CSE**

Sign with Date:
Ms. T. Tharunika
(20WH1A0542)

ACKNOWLEDGEMENT

We would like to express our sincere thanks to **Dr. K. V. N. Sunitha, Principal, BVRIT HYDERABAD College of Engineering for Women**, for her support by providing the working facilities in the college.

Our sincere thanks and gratitude to **Dr. R. Suneetha Rani , Head, Department of CSE, BVRIT HYDERABAD College of Engineering for Women**, for all the timely support and valuable suggestions during the period of our project.

We are extremely thankful to our Internal Guide, **Ms. S. Vidyullatha, Assistant Professor, CSE, BVRIT HYDERABAD College of Engineering for Women**, for her constant guidance and encouragement throughout the project.

Finally, we would like to thank our Mini Project Coordinator and all Faculty and Staff of the CSE department who helped us directly or indirectly. Last but not least, we wish to acknowledge our **Parents** and **Friends** for giving us moral strength and constant encouragement.

T. Tharunika (20WH1A0542)

ABSTRACT

Music plays a very important role in people's lives. Music brings like-minded people together and is the glue that holds communities together. Communities can be recognized by the type of songs that they compose, or even listen to. The purpose of our project and research is to find a better machine learning and Deep learning algorithms than the pre-existing models that predicts the genre of songs. In this project, we built multiple classification models and trained them over the GTZAN dataset. We have compared the performances of all these models and logged their results in terms of prediction accuracies. Few of these models are trained on the mel-spectrograms of the songs along with their audio features, and few others are trained solely on the spectrograms of the songs. It is found that one of the models, a convolutional neural network, which has given spectrograms as the dataset, has given the highest accuracy amongst all other models.

LIST OF FIGURES

S. No	Description	Page. No
1	Workflow	11
2	Confusion matrix for transfer model	23
3	Confusion matrix for CNN model	23

LIST OF CONTENTS

S. No	Topic	Page No.
1	Introduction	8
2	Requirements	10
3	Design	11
4	Implementation	12
5	Testing	25
6	Conclusion & Future scope	26
7	References	27

1.INTRODUCTION

Sound is represented in the form of an audio signal having parameters like frequency, decibel, bandwidth etc. A typical audio signal can be expressed as a function of Amplitude and Time. These audio signals come in various different formats which make it possible and easy for the computer to read and analyse them. Some formats are: mp3 format, WMA (Windows Media Audio) format and wav (Waveform AudioFile) format. Companies (such as Soundcloud, Apple Music, Spotify, Wynk etc) use music classification, either to place recommendations to their customers, or simply as a product. To be able to do any of the above two mentioned functions, determining music genres is the first step. To achieve this, we can take the help of Machine Learning and Deep Learning models. These models prove to be very handy in Music Analysis too. Music Analysis is done based on a song's features to determine the kind of songs that a person is interested to listen. Music is characterized by giving them categorical labels called genres. These genres are created by humans. A music genre is segregated by the characteristics which are commonly shared by its members. Typically, these characteristics are related to the rhythmic structure, instrumentation, and the harmonic content of the music. To categorize music files into their respective genres, it is a very challenging task in the area of Music Information Retrieval (MIR), a field concerned with browsing, organizing and searching large music collections. Classification of genre can be very valuable to explain some interesting problems such as creating song references, tracking down related songs, discovering societies that will like that specific song, sometimes it can also be used for survey purposes. Automatic musical genre classification can assist humans or even replace them in this process and would be of a very valuable addition to music information retrieval systems. In addition to this, automatic classification of music into genres can provide a framework for development and evaluation of features for any type of content-based analysis of musical signals. The concept of automatic music genre classification has become very popular in recent years as a result of the rapid growth of the digital entertainment industry. Dividing music into genres is arbitrary, but there are perceptual criteria that are related to instrumentation, structure of the rhythm and texture of the music that can play a role in characterizing genre. Until now genre classification for digitally available music has been performed manually. Thus, techniques for automatic genre classification would be a valuable addition to the development of audio information retrieval systems for music.

1.1 Problem Statement:

Music genre classification model is a model to classify music based on variety of features into its corresponding genre. This project aims at developing machine learning and deep learning models and train them on a dataset of audio features extracted from a large collection of music tracks. The models are designed and developed to produce better accuracy for music genre classification.

1.2 Objectives:

1. Developing a machine learning model that classifies music into genres that shows there exists a solution which automatically classifies music into its genres based on various different features, instead of manually entering the genre.
2. Another objective is to reach a good accuracy so that the model classifies new music into its genre correctly.
3. This model is expected to perform better than at least few pre-existing models.

1.3 Proposed System:

Our team initiated the development of music genre classification system utilizing a dataset consisting of various features of audio tracks. The key components involved are:

1.3.1 Dataset Collection:

We gathered the dataset from Kaggle. We started by compiling a dataset containing data regarding various features of audio tracks like filename, audiolength, chroma_stft_mean, chroma_stft_var, rms_mean, rms_var, spectral_centroid_mean, spectral_centroid_var, spectral_bandwidth_mean, spectral_bandwidth_var, rolloff_mean, rolloff_var, zero_crossing_rate_mean, zero_crossing_rate_var, harmony_mean, harmony_var, perceptr_mean, perceptr_var, tempo, mfcc's, labels etc. This dataset formed the basis for training and evaluating our music genre classification model.

1.3.2 Data Cleaning:

There are no missing values or null values or duplicate values in the dataset.

1.3.3 Data Preprocessing:

We have categorical data for class labels so we converted them into numerical data using Label Encoder. We have done feature extraction for machine learning models using PCA (principle Component Analysis).

2. REQUIREMENTS

2.1 Software requirements:

- Windows 10
- Google Colab
- Python 3.8
- ipywidgets

2.2 Hardware Requirements:

- Intel core i3 processor
- RAM 4GB
- Hard Disk Drive 1 TB

3.DESIGN

3.1 Architecture:

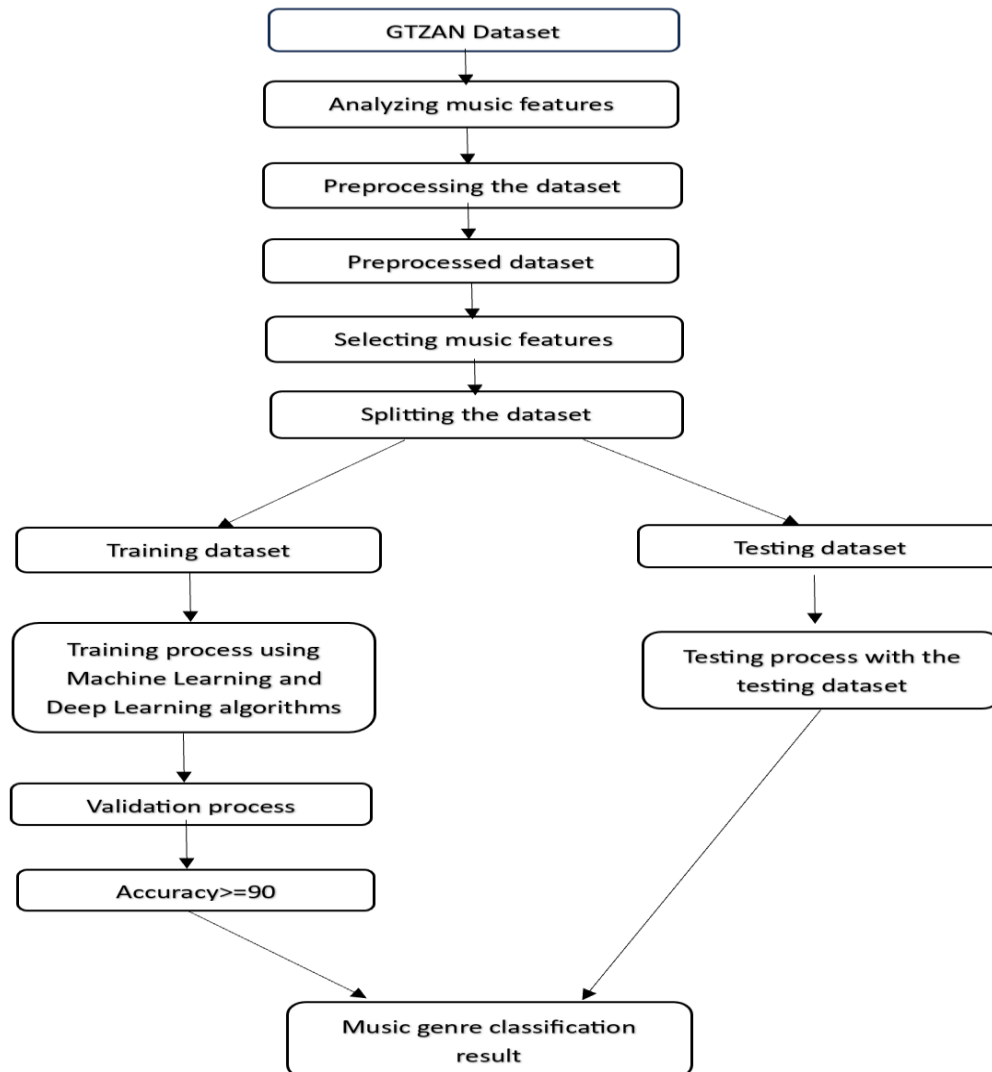


Figure 3.1.1 Music Genre Classification system workflow

4. IMPLEMENTATION

Main Code:

4.1 Code using dataset:

4.1.1 Dataset Loading:

```
import pandas as pd
import numpy as np
df = pd.read_csv("/content/drive/MyDrive/MLProject/miniprodataset.csv")
```

```
df.head()
```

O/P:

	filename	length	chroma_stft_mean	chroma_stft_var	rms_mean	rms_var	spectral_centroid_mean	spectral_centroid_var	spectral_bandwidth
0	blues.00000.0.wav	66149	0.335406	0.091048	0.130405	0.003521	1773.065032	167541.630869	1972.744388
1	blues.00000.1.wav	66149	0.343065	0.086147	0.112699	0.001450	1816.693777	90525.690866	2010.051501
2	blues.00000.2.wav	66149	0.346815	0.092243	0.132003	0.004620	1788.539719	111407.437613	2084.565132
3	blues.00000.3.wav	66149	0.363639	0.086856	0.132565	0.002448	1655.289045	111952.284517	1960.039988
4	blues.00000.4.wav	66149	0.335579	0.088129	0.143289	0.001701	1630.656199	79667.267654	1948.503884

4.1.2 Data Cleaning:

```
df.duplicated().sum()
df.drop_duplicates
df.isnull().sum()
```

4.1.3 Data Preprocessing:

```
from sklearn.preprocessing import LabelEncoder
labelencoder = LabelEncoder()
df['label'] = labelencoder.fit_transform(df['label'])
print(df['label'])
```

O/P:

```
0    0
1    0
2    0
3    0
4    0
...
995   9
996   9
997   9
998   9
999   9
Name: label, Length: 1000, dtype: int64
```

```
df.drop(['filename'], axis=1, inplace=True)
df.head()
```

O/P:

	length	chroma_stft_mean	chroma_stft_var	rms_mean	rms_var	spectral_centroid_mean	spectral_centroid_var	spectral_bandwidth_mean	spectral
0	66149	0.335406	0.091048	0.130405	0.003521	1773.065032	167541.630869	1972.744388	
1	66149	0.343065	0.086147	0.112699	0.001450	1816.693777	90525.690866	2010.051501	
2	66149	0.346815	0.092243	0.132003	0.004620	1788.539719	111407.437613	2084.565132	
3	66149	0.363639	0.086856	0.132565	0.002448	1655.289045	111952.284517	1960.039988	
4	66149	0.335579	0.088129	0.143289	0.001701	1630.656199	79667.267654	1948.503884	

```
from sklearn.decomposition import PCA
pca = PCA(whiten=True)
```

```
pca.fit(df)
variance = pd.DataFrame(pca.explained_variance_ratio_)
np.cumsum(pca.explained_variance_ratio_)
pca = PCA(n_components=25,whiten=True)
pca = pca.fit(df)
dataPCA = pd.DataFrame(pca.transform(df))
X= dataPCA
Print(X)
```

O/P:

```
      0      1      2      3      4      5      6  \
0 -0.384945 -0.448064  0.593063 -0.005834 -0.135440  0.760913  0.431983
1 -0.657655 -0.371607  0.001288  0.062999 -0.080034  0.525601  0.023177
2 -0.579004 -0.397729  0.046037  0.729340  0.336703  0.816512 -0.539265
3 -0.495539 -0.508474  0.008751  0.225841 -0.215638  0.495360 -0.649567
4 -0.698848 -0.357525 -0.033522  0.176040 -0.346402  0.753950 -0.272179
...
9985 -0.534381 -0.255316  0.257667  0.063323 -0.846845  0.113792 -0.590286
9986 -0.311507 -0.104438  0.174411 -0.585143 -0.758746 -0.063262 -0.529257
9987  0.655640  0.072245 -0.846228  6.536782  0.993021  0.931798  0.702187
9988 -0.753645  0.202332 -0.554363  0.628409  0.076124  0.321115 -0.587811
9989 -0.071652  0.075031  0.210195 -0.191008 -1.030733 -1.042455 -1.030936
```

4.1.4 Splitting of dataset:

```
y = df.iloc[:, -1]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.20, random_state=42)
```

4.1.5 DecisionTree Classifier:

A decision tree classifier is a supervised learning algorithm used for classification tasks. It works by recursively partitioning the input space into smaller regions or categories based on feature values.

```
from sklearn.metrics import confusion_matrix,
accuracy_score, classification_report
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier().fit(X_train, y_train)
y_pred = clf.predict(X_test)
print(accuracy_score(y_pred, y_test)*100)
```

O/P:

```
48.44844844844845
```

4.1.6 Random Forest:

It is an ensemble learning method based on the concept of decision trees. It builds multiple decision trees and merges their predictions to improve accuracy and reduce overfitting.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix,
accuracy_score, classification_report
rf= RandomForestClassifier(n_estimators=100,random_state=0)
rf.fit(X_train,y_train)
y_pred= rf.predict(X_test)
print("Accuracy Score RandomForest: ",accuracy_score(y_test,y_pred))
```

O/P:

Accuracy Score RandomForest: 0.6921921921921922

4.1.7 Multinomial Logistic Regression:

It is a classification method that generalizes logistic regression to multiclass problems. It is a model that is used to predict the probabilities of different possible outcomes of a categorically distributed dependent variable.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
model = LogisticRegression(multi_class='multinomial', solver='lbfgs',
max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy Score: ", accuracy_score(y_pred, y_test))
```

O/P:

Accuracy Score: 0.549049049049049

4.1.8 XG Boost:

XGBoost is a scalable and highly accurate implementation of gradient boosting that pushes the limits of computing power for boosted tree algorithms, being built largely for energizing machine learning model performance and computational speed.

```
from xgboost import XGBClassifier
model = XGBClassifier(n_estimators=1000)
model.fit(X_train, y_train, eval_metric='merror')
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)
target_names = sorted(set(y))

print(f'Training accuracy: {accuracy_score(y_train, y_pred_train)}')
#print(f'Training:\n {classification_report(y_train, y_pred_train,
labels=target_names)}')
print(f'Testing accuracy: {accuracy_score(y_test, y_pred_test)}')
#print(f'Testing:\n {classification_report(y_test, y_pred_test,
labels=target_names)}')
```

O/P:

Training accuracy: 0.9998748748748749

Testing accuracy: 0.7267267267267268

4.2 Code with audio and image files:**4.2.1 Convolutional Neural Network:**

A convolutional neural network, or CNN, is a deep learning neural network designed for processing structured arrays of data such as images. Convolutional neural networks are widely used in computer vision and have become the state of the art for many visual

applications such as image classification, and have also found success in natural language processing for text classification.

```
import random, os, glob # default python modules that let me randomise
and manipulate files
import numpy as np # for data manipulation through arrays
import tensorflow as tf
from keras.models import Sequential # the model I will use
from keras.layers import Dropout, Dense, Conv2D, MaxPool2D, Flatten,
Reshape, BatchNormalization, GlobalAveragePooling2D # layers I will
incorporate
from keras.callbacks import EarlyStopping # for better training
from tensorflow.keras.applications import VGG19 # transfer learning model
from keras import backend
from livelossplot import PlotLossesKeras # to visually display how my model
improves as training progresses
import librosa # to demonstrate the creation of a mel spectrogram
from librosa.display import specshow
import matplotlib.pyplot as plt
import IPython.display as ipd
from sklearn.metrics import confusion_matrix
import seaborn as sns
def setRandom():
    seed = 0 # random seed value
    os.environ["PYTHONHASHSEED"] = str(seed) # if this is not set, a
random value is used to seed the hashes of some objects
    random.seed(seed) # sets the base python and numpy random seeds
    np.random.seed(seed)
    tf.random.set_seed(seed) # sets the tensorflow random seed
    tf.compat.v1.set_random_seed(seed)
filePath = "/content/drive/MyDrive/GTZAN/genres/disco/disco.00001.wav" #
an example file

file, samplingRate = librosa.load(filePath)
example, _ = librosa.effects.trim(file)

hopLength = 512 # the number of samples between successive columns of
the spectrogram

spectrogram = librosa.power_to_db(librosa.feature.melspectrogram(y =
example, sr = samplingRate, n_fft = 2048, hop_length = hopLength,
n_mels = 128, power = 4.0), ref = np.max)

plt.figure(figsize = (3, 2))
librosa.display.specshow(spectrogram, sr = samplingRate, hop_length =
hopLength, x_axis = "off", y_axis = "off")
ipd.Audio(example, rate = samplingRate)
```

```

source = "/content/drive/MyDrive/GTZAN/images_original"# source folder
path
genres = ["blues", "classical", "country", "disco", "hiphop", "jazz",
"metal", "pop", "reggae", "rock"] # list with the genre folder names

for genre in genres: # iterate through each genre folder
    path = os.path.join(source, genre)
    pngs = [ifori in os.listdir(path) if i[-4:] == ".png"] # get a list of
.png files in the genre folder
    print(f"Size of {genre} dataset: {len(pngs)} files.")

setRandom()
split = [80, 9, 10]
train, val, test = {}, {}, {} # empty dictionaries to store the
filepaths
trainLen, valLen, testLen = {}, {}, {} # empty dictionaries to store
the number of files under each genre for each dataset
dictionaries = [train, val, test]

for d in dictionaries:
    if d == train: num = slice(0, split[0])
    elif d == val: num = slice(split[0], split[0] + split[1])
    else: num = slice(split[0] + split[1], split[0] + split[1] +
split[2])
    for genre in genres: # iterate through each genre folder
        path = os.path.join(source, genre)
        pngs = glob.glob(os.path.join(path, "*.png")) # get a list of
.png filepaths in the genre folder
        selected = pngs[num] # take the first 80 files
        d[genre] = selected # store the selected files in the
dictionary

lenDictionaries = [{genre: len(d[genre]) for genre in genres} for d in
dictionaries]

print(f"\033[1mTraining:\033[0m {lenDictionaries[0]}")
print(f"\033[1mValidation:\033[0m {lenDictionaries[1]}")
print(f"\033[1mTest:\033[0m {lenDictionaries[2]}")

batchSize = 512# typical batch size for a neural network
genreMap = {
    "blues": 0,
    "classical": 1,
    "country": 2,
    "disco": 3,
    "hiphop": 4,
    "jazz": 5,
    "metal": 6,

```



```

    "pop": 7,
    "reggae": 8,
    "rock": 9
}
inverseGenreMap = {value: key for key, value in genreMap.items()}

def createDataset(d):
    imgSize = (288, 432) # define image and batch parameters
    imageList, labelList = [], [] # create lists to store images and
    labels

    for genre, paths in d.items():
        for path in paths:
            image =
            tf.cast(tf.image.resize(tf.image.decode_png(tf.io.read_file(path),
            channels = 3), imgSize), tf.float32) / 255.0 # normalise pixel values
            between 0 and 1 (preprocessing!)
            imageList.append(image)
            labelList.append(genreMap[genre]) # convert genre to its
            integer label

    dataset = tf.data.Dataset.from_tensor_slices((imageList,
    labelList)).shuffle(buffer_size=len(imageList)).batch(batchSize) #
    create and return tensorflow dataset
    return dataset

def prep(ds):
    out = (
        ds.map(lambda image, label:
        (tf.image.convert_image_dtype(image, tf.float32), label)) # modifies
        the image tensor's data type to floats
        .cache() # cache dataset elements in memory or on disk to speed
        up data loading
        .prefetch(buffer_size = tf.data.experimental.AUTOTUNE) #
        prefetch dataset elements in the background and automatically optimise
        data loading
    )
    return out # return the prepared and optimised dataset

training, validation, testing = prep(createDataset(train)),
prep(createDataset(val)), prep(createDataset(test))
print("Datasets created.") # to let me know when it's finished running
(:

def view_dataset(dataset):
    genreExamples = {} # dictionary to store examples for each label

    for images, labels in dataset:

```

```

        for image, label in zip(images, labels):
            label = int(label.numpy()) # convert label tensor to
integer
            if label not in genreExamples:
                genreExamples[label] = image

            if len(genreExamples) == len(genres):
                break

# display the randomly chosen examples
plt.figure(figsize = (30, 20))
for label, image in genreExamples.items():
    ax = plt.subplot(1, len(genres), label + 1)
    plt.imshow(image)
    plt.title(inverseGenreMap[label])
    plt.axis("off")
plt.show()

print("\033[1mTraining Examples:\033[0m"); view_dataset(training) #
shows a labelled example of a mel spectrogram from each genre from each
dataset
print("\033[1mValidation Examples:\033[0m"); view_dataset(validation)
print("\033[1mTesting Examples:\033[0m"); view_dataset(testing)
cnn = Sequential([
    BatchNormalization(input_shape = inputShape),

    Conv2D(32, (3, 3), activation = "LeakyReLU"),
    MaxPool2D((2, 2)),

    Conv2D(64, (3, 3), activation = "LeakyReLU"),
    MaxPool2D((2, 2)),

    Conv2D(128, (3, 3), activation = "LeakyReLU"),
    MaxPool2D((2, 2)),

    Conv2D(256, (3, 3), activation = "LeakyReLU"),
    MaxPool2D((2, 2)),

    Conv2D(512, (3, 3), activation = "LeakyReLU"),
    MaxPool2D((2, 2)),

    Flatten(),
    Dense(1024, activation = "LeakyReLU"),
    Dropout(0.5),
    Dense(512, activation = "LeakyReLU"),
    Dropout(0.5),
    BatchNormalization(),
    Dense(len(genres), activation = "softmax")
])

```

```

])

cnn.compile(optimizer = tf.keras.optimizers.SGD(lr = 0.001), loss =
"sparse_categorical_crossentropy", metrics = ["accuracy"]) #
"sparse_categorical_crossentropy" because labels are integers
cnn.summary()

cnn.fit(training, validation_data = validation, batch_size = 32, epochs
= 100, verbose = 1, callbacks = [earlyStopping, PlotLossesKeras()])

```

O/P:

WARNING:absl:lr is deprecated in Keras optimizer, please use 'learning_rate' or use the legacy optimizer, e.g., tf.keras.optimizers.lsg

Model: "sequential_6"

Layer (type)	Output Shape	Param #
batch_normalization_198 (Batch Normalization)	(None, 288, 432, 3)	12
conv2d_288 (Conv2D)	(None, 288, 432, 32)	896
max_pooling2d_28 (MaxPooling2D)	(None, 144, 216, 32)	0
conv2d_289 (Conv2D)	(None, 144, 216, 64)	18496
max_pooling2d_29 (MaxPooling2D)	(None, 72, 108, 64)	0
conv2d_218 (Conv2D)	(None, 68, 104, 128)	73856
max_pooling2d_30 (MaxPooling2D)	(None, 34, 52, 128)	0
conv2d_211 (Conv2D)	(None, 32, 50, 256)	295168
max_pooling2d_31 (MaxPooling2D)	(None, 16, 25, 256)	0
conv2d_212 (Conv2D)	(None, 14, 23, 512)	1180160
max_pooling2d_32 (MaxPooling2D)	(None, 7, 11, 512)	0
flatten_6 (Flatten)	(None, 39424)	0
dense_20 (Dense)	(None, 1824)	48371200
dropout_14 (Dropout)	(None, 1824)	0
dense_21 (Dense)	(None, 512)	524800
dropout_15 (Dropout)	(None, 512)	0
batch_normalization_199 (Batch Normalization)	(None, 512)	2048
dense_22 (Dense)	(None, 10)	5130

Total params: 42471766 (162.02 MB)
Trainable params: 42470736 (162.01 MB)
Non-trainable params: 1030 (4.02 KB)

4.2.2 Transfer Model:

Transfer learning is the method of starting with a pre-trained model and training it for a new – related problem domain. The pre-trained network serves as transferred knowledge to be applied in another domain. But there are numerous options that can be used, including feature transfer and fine-tuning (which depend upon the similarity of the problems at hand), in addition to freezing certain layers of the network and retraining others.

```

inputShape = [288, 432, 3] # the shape of the images (288px tall, 432px
wide, and 3 colour channels/RGB)

earlyStopping = EarlyStopping( # a custom early stopping setup to
automatically stop training when loss doesn't increase enough after a
number of epochs
    min_delta = 0.001, # minimum amount of change to count as an
improvement
    patience = 20, # how many epochs to wait before stopping
    restore_best_weights = True # tells it to restore back to when loss
was at its lowest value
)

```

```

from tensorflow.keras.applications import InceptionV3 # transfer learning
model
baseModel = InceptionV3(input_shape = inputShape, weights = "imagenet",
include_top = False, pooling = "avg")

for layer in baseModel.layers:
    layer.trainable = False # freeze the pre-trained layers

transfer = Sequential([
    baseModel,

    Flatten(),
    BatchNormalization(),
    Dense(512, activation = "LeakyReLU"),
    Dropout(0.3),
    Dense(256, activation = "LeakyReLU"),
    Dropout(0.3), # dropout layer to prevent overfitting
    Dense(128, activation = "LeakyReLU"),
    Dropout(0.3),
    Dense(len(genres), activation = "softmax")
])

transfer.compile(optimizer = tf.keras.optimizers.SGD(lr = 0.0001), loss
= "sparse_categorical_crossentropy", metrics = ["accuracy"])
transfer.summary()
transferHistory = transfer.fit(training, validation_data = validation,
batch_size = 256, epochs = 500, verbose = 1, callbacks =
[earlyStopping, PlotLossesKeras()])

```

O/P:

WARNING:absl:lr is deprecated in Keras optimizer, please use 'learning_rate' or use the legacy optimizer, e.g., tf.keras.optimizers.legacy.Adam

Layer (type)	Output Shape	Param #
inception_v3 (Functional)	(None, 2048)	21862784
flatten_3 (Flatten)	(None, 2048)	0
batch_normalization_193 (Batch Normalization)	(None, 2048)	8192
dense_18 (Dense)	(None, 512)	1049088
dropout_7 (Dropout)	(None, 512)	0
dense_11 (Dense)	(None, 256)	131328
dropout_8 (Dropout)	(None, 256)	0
dense_12 (Dense)	(None, 128)	32896
dropout_9 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 10)	1290
Total params: 23025578 (87.84 MB)		
Trainable params: 1218698 (4.65 MB)		
Non-trainable params: 21806880 (83.19 MB)		

```

def confusionMatrix(model, name):
    trueLabels = np.concatenate([y for x, y in testing], axis = 0) #
    get the true labels from the testing dataset

```

```

    predictedLabels = np.argmax(model.predict(testing, verbose = 0),
axis = 1) # get the predicted labels from the model

    matrix = confusion_matrix(trueLabels, predictedLabels) # create the
confusion matrix

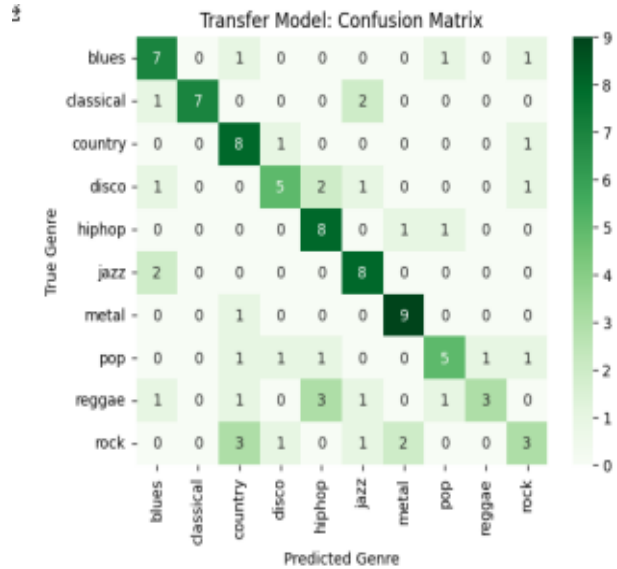
    plt.figure() # plot the confusion matrix using seaborn for the
heatmap
    sns.heatmap(matrix, annot = True, cmap = "Greens", xticklabels =
genres, yticklabels = genres)
    plt.xlabel("Predicted Genre")
    plt.ylabel("True Genre")
    plt.title(f"{name} Model: Confusion Matrix")
    plt.show()

    trainStats, valStats, testStats = model.evaluate(training, verbose
= 0), model.evaluate(validation, verbose = 0), model.evaluate(testing,
verbose = 0)
    print(f"\033[1m{name} Model\033[0m")
    print(f"Training Accuracy: {trainStats[1]}% \nTrain Loss:
{trainStats[0]}\n")
    print(f"Validation Accuracy: {valStats[1]}% \nTest Loss:
{valStats[0]}\n")
    print(f"Testing Accuracy: {testStats[1]}% \nTest Loss:
{testStats[0]}")

confusionMatrix(transfer, "Transfer")
confusionMatrix(cnn, "Custom CNN")

```

O/P:



Transfer Model

Training Accuracy: 0.9787499904632568%

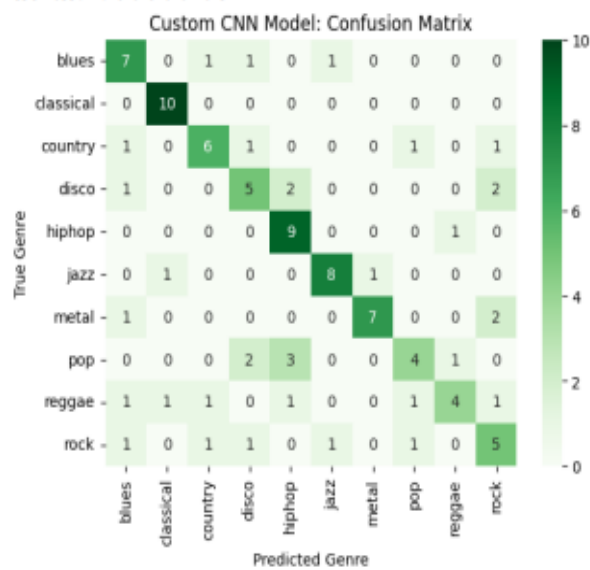
Train Loss: 0.2164267599582672

Validation Accuracy: 0.6666666865348816%

Test Loss: 0.9666965007781982

Testing Accuracy: 0.6299999952316284%

Test Loss: 1.181628704071045



Custom CNN Model

Training Accuracy: 1.0%

Train Loss: 0.05530594661831856

Validation Accuracy: 0.6666666865348816%

Test Loss: 1.06783127784729

Testing Accuracy: 0.6499999761581421%

Test Loss: 1.082409381866455

4.3 Comparison Table:

4.3.1 Models implemented for Dataset:

S. No	Models	Accuracy
1	Decision Tree	48.44%
2	Multinomial Logistic Regression	54.90%
3	Random Forest	69.21%
4	XG Boost	72.67%

4.3.2 Models implemented for Audio files:

S. No	Models	Accuracy
1	Transfer	62.99%
2	CNN	64.99%

5. TESTING

```
import ipywidgets as widgets
from IPython.display import display
import tensorflow as tf
import numpy as np

# Create an interactive widget using ipywidgets interact
image_path_widget = widgets.Text(
    value='path/to/your/audio',
    description='Audio Path:',
    layout=widgets.Layout(width='50%') # Adjust the width of the input
    box
)

output_widget = widgets.Output()

def on_button_click(b):
    with output_widget:
        output_widget.clear_output(wait=True)
        predicted_genre = predict_audio(image_path_widget.value)
        print(f"Model Prediction: {predicted_genre}")

button = widgets.Button(description="Predict")
button.on_click(on_button_click)

# Display the widgets
display(image_path_widget)
display(button)
display(output_widget)
```

O/P:

Audio Path:

Predict

```
1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 24ms/step
Model Prediction: classical
```


6. CONCLUSION & FUTURE SCOPE

After entering the digital era, digital music technology has led to the rise of Internet companies. In this work, the task of music genre classification is studied using the GTZAN dataset. We propose two different approaches to solve this problem. The first approach was to improve the accuracy using machine learning algorithms like Decision tree, Random Forest, Multinomial Logistic Regression, XGBoost which were determined to be the best feature-based classifiers; the most important features were also reported. Secondly, we used deep learning models like CNN & Transfer model, which were shown to outperform the feature-engineered models. This will fundamentally change the way music content reaches music users and is a disruptive technology application for the industry.

7. REFERENCES

- Tom LH Li, Antoni B Chan, and A Chun. 2010. Automatic musical pattern feature extraction using convolutional neural network. InProc. Int. Conf. Data Mining and Applications.
- Loris Nanni, Yandre MG Costa, Alessandra Lumini, MooYoung Kim, and Seung Ryul Baek. 2016. Combining visual and acoustic features for music genre classification. *Expert Systems with Applications* 45:108–117.
- Hareesh Bahuleyan, (2018). Music Genre Classification using Machine Learning techniques.
- Prajwal, R., et al. “Music genre classification using machine learning.” *Int. Res. J. Mod. Eng. Technol. Sci* 3.7 (2021): 953-957.
- Fan, Minglei. “Application of music industry based on the deep neural network.” *Scientific Programming* 2022 (2022): 1-6.
- Shah, Mitt, et al. “Music genre classification using deep learning.” 2022 6th International Conference on Computing Methodologies Communication (ICCMC). IEEE, 2022.